~\Desktop\C++学习资料\Untitled-1.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
struct Edge { int u, v, s, t, id; } E[M];
vector<int> g[N];
int d[N], w[M]n, m, S, T, start_time;   // d:到达站点i的最早时间, w:坐完第j张票的最小等待
int solve_time() {
    memset(d, 0x3f, sizeof(d));
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> q;
    d[S] = start_time;q.push({d[S], S});
    while(q.size()) {
        int time = q.top().first, u = q.top().second; q.pop();
        if(time > d[u]) continue;
        if(u == T) return time - start_time; // 找到即返回(也可跑完)
        for(int id : g[u])
            if(E[id].s >= time && E[id].t < d[E[id].v]) {
                d[E[id].v] = E[id].t;
                q.push({d[E[id].v], E[id].v});
            }
    }return -1;
}
int solve_wait() {
    memset(w, 0x3f, sizeof(w));int ans = INF;
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> q;
    for(int id : g[S])
        if(E[id].s >= start_time) {
            w[id] = E[id].s - start_time;
            q.push({w[id], id});
        }
    while(q.size()) {
        int wait = q.top().first, id = q.top().second; q.pop();
        if(wait > w[id]) continue;
        if(E[id].v == T) ans = min(ans, wait);// 如果这条边的终点是T，尝试更新答案
        for(int nxt : g[E[id].v])// 尝试接下一班车
            if(E[nxt].s >= E[id].t) {
                int cost = E[nxt].s - E[id].t; // 中转等待
                if(wait + cost < w[nxt]) {
                    w[nxt] = wait + cost;
                    q.push({w[nxt], nxt});
                }
            }
    }return ans == INF ? -1 : ans;
}
int main() {
    ios::sync_with_stdio(0); cin.tie(0);
    cin >> n >> m >> S >> T >> start_time;
    for(int i = 0; i < m; i++) {
        cin >> E[i].u >> E[i].v >> E[i].s >> E[i].t;
        E[i].id = i;g[E[i].u].push_back(i);
    }
    int t1 = solve_time(),t2 = solve_wait();
}
```